



OpenShift Container Platform 4.22

Virtualized control planes

Deploying OpenShift Container Platform clusters with virtualized control planes

OpenShift Container Platform 4.22 Virtualized control planes

Deploying OpenShift Container Platform clusters with virtualized control planes

Legal Notice

Copyright © Red Hat.

Except as otherwise noted below, the text of and illustrations in this documentation are licensed by Red Hat under the Creative Commons Attribution–Share Alike 3.0 Unported license . If you distribute this document or an adaptation of it, you must provide the URL for the original version.

Red Hat, as the licensor of this document, waives the right to enforce, and agrees not to assert, Section 4d of CC-BY-SA to the fullest extent permitted by applicable law.

Red Hat, the Red Hat logo, JBoss, Hibernate, and RHCE are trademarks or registered trademarks of Red Hat, LLC. or its subsidiaries in the United States and other countries.

Linux[®] is the registered trademark of Linus Torvalds in the United States and other countries.

XFS is a trademark or registered trademark of Hewlett Packard Enterprise Development LP or its subsidiaries in the United States and other countries.

The OpenStack[®] Word Mark and OpenStack logo are trademarks or registered trademarks of the Linux Foundation, used under license.

All other trademarks are the property of their respective owners.

Abstract

This document describes how to deploy an OpenShift Container Platform cluster with control plane nodes running as virtual machines on a hosting cluster, by using KubeVirt Redfish and OpenShift Virtualization.

Table of Contents

CHAPTER 1. UNDERSTANDING VIRTUALIZED CONTROL PLANES	3
1.1. VIRTUALIZED CONTROL PLANE ARCHITECTURE	3
1.2. VIRTUALIZED CONTROL PLANE DEPLOYMENT WORKFLOW	4
1.3. ADDITIONAL RESOURCES	4
CHAPTER 2. PREREQUISITES FOR VIRTUALIZED CONTROL PLANES	5
2.1. HOSTING CLUSTER REQUIREMENTS	5
2.2. STORAGE REQUIREMENTS	5
2.3. NETWORK REQUIREMENTS	6
2.4. CONTROL PLANE VM REQUIREMENTS	6
2.5. KUBEVIRT REDFISH REQUIREMENTS	6
2.6. ADDITIONAL RESOURCES	7
CHAPTER 3. PREPARING THE ENVIRONMENT FOR VIRTUALIZED CONTROL PLANES	8
3.1. INSTALL KUBEVIRT REDFISH	8
3.2. CREATE CONTROL PLANE VMS	15
3.3. ADDITIONAL RESOURCES	18
CHAPTER 4. DEPLOYING A VIRTUALIZED CONTROL PLANE	19
4.1. DEPLOYING A VIRTUALIZED CONTROL PLANE WITH THE AGENT-BASED INSTALLER	19
4.2. DEPLOYING A VIRTUALIZED CONTROL PLANE WITH GITOPS ZTP	24
4.3. DEPLOYING A VIRTUALIZED CONTROL PLANE WITH INSTALLER-PROVISIONED INFRASTRUCTURE	27
4.4. ADDITIONAL RESOURCES	31

CHAPTER 1. UNDERSTANDING VIRTUALIZED CONTROL PLANES

A virtualized control plane deployment is an OpenShift Container Platform cluster whose control plane nodes run as virtual machines (VMs) on a hosting cluster with OpenShift Virtualization.



IMPORTANT

KubeVirt Redfish is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

This architecture is useful in the following example scenarios:

- Regulatory requirements mandate VM-level isolation for control plane components.
- You want to reduce hardware costs by consolidating multiple cluster control planes on shared infrastructure.
- You need faster provisioning of new clusters compared to physical bare metal.

In a virtualized control plane deployment, you have two clusters:

Hosting cluster

An existing OpenShift Container Platform cluster running OpenShift Virtualization that hosts the control plane VMs.

Target cluster

The OpenShift Container Platform cluster with control planes running on the VMs.

KubeVirt Redfish runs on the hosting cluster and exposes the VMs through the standard Redfish API endpoints.

With this approach, you can use installation workflows such as Agent-based Installer or GitOps Zero Touch Provisioning (ZTP), to deploy virtualized control planes exactly like physical servers with baseboard management controllers (BMCs).



NOTE

Virtualized control planes differ from Hosted control planes. With virtualized control planes, the control plane runs as VMs with hypervisor-level isolation. With Hosted control planes, the control plane runs as pods with container-level isolation.

1.1. VIRTUALIZED CONTROL PLANE ARCHITECTURE

A virtualized control plane deployment runs control plane components as VMs on a hosting cluster, providing hypervisor-level isolation between clusters.

A single hosting cluster can support multiple target clusters by running each cluster's control plane VMs

in separate namespaces. This consolidation reduces hardware costs while maintaining isolation. The target cluster's worker nodes run on separate infrastructure, either physical servers or VMs on different hosts.

For high availability, distribute control plane VMs across different physical nodes on the hosting cluster. This anti-affinity placement ensures that if a physical node fails, only one control plane VM is affected and the remaining nodes maintain etcd quorum. Configure anti-affinity using pod anti-affinity rules or topology spread constraints in the VM specifications.

1.2. VIRTUALIZED CONTROL PLANE DEPLOYMENT WORKFLOW

Deploy a virtualized control plane cluster by installing KubeVirt Redfish on your hosting cluster, configuring it to expose your VMs, and running your preferred installation method.



NOTE

Virtualized control planes require an OpenShift Container Platform cluster with OpenShift Virtualization installed and operational, which operates as the hosting cluster.

See the following high-level steps to deploy a virtualized control plane cluster:

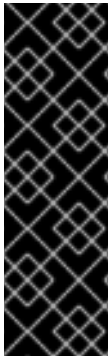
1. Install and configure KubeVirt Redfish on the hosting cluster. This includes defining which VMs to expose through the Redfish API, configuring authentication credentials, and creating a **Route** CR to expose the endpoint externally.
2. Create the control plane VMs on the hosting cluster. Configure the VMs with appropriate resources and network settings, and ensure they remain powered off until the installation begins.
3. Configure your installation method to use KubeVirt Redfish. In your configuration files, specify BMC addresses using the KubeVirt Redfish route URL for the virtualized control plane nodes, for example: **redfish-virtualmedia+https://<kubevirt_redfish_route>/redfish/v1/Systems/<vm_namespace>.<vm_name>**.
4. Run the installation. The VMs boot from the installation media and communicate with each other to form the cluster. Depending on the installation method, this process is either fully automated or requires manual intervention to boot each node.
5. After installation completes, a new OpenShift Container Platform cluster is deployed with its control plane running on VMs hosted by the original OpenShift Virtualization cluster.

1.3. ADDITIONAL RESOURCES

- [Prerequisites for virtualized control planes](#)
- [Installing KubeVirt Redfish](#)
- [Configuring KubeVirt Redfish for VM management](#)
- [BMC addressing for installer-provisioned infrastructure](#)
- [Deploying far edge sites with ZTP](#)

CHAPTER 2. PREREQUISITES FOR VIRTUALIZED CONTROL PLANES

Before deploying a virtualized control plane cluster, ensure your environment meets the following requirements.



IMPORTANT

KubeVirt Redfish is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

2.1. HOSTING CLUSTER REQUIREMENTS

The hosting cluster runs the virtualized control plane VMs and manages their lifecycle through KubeVirt Redfish.

The hosting cluster requires the following components:

- OpenShift Container Platform installed and operational.
- OpenShift Virtualization installed and configured.
- Sufficient compute resources to host control plane VMs.
- At least three physical nodes to enable anti-affinity placement, ensuring control plane VMs are spread across different physical hosts.

2.2. STORAGE REQUIREMENTS

The hosting cluster must have a storage solution that supports VM disk images and meets the latency requirements of **etcd**.

The hosting cluster must have one of the following storage solutions:

- Red Hat OpenShift Data Foundation (ODF) provides VM live migration. When you install ODF on a cluster where OpenShift Virtualization is running, the **ocs-storagecluster-ceph-rbd-virtualization** storage class is created automatically. This storage class is optimized for KubeVirt workloads.
- Logical Volume Manager (LVM) Storage, backed by NVMe drives, provides predictable, low-latency disk I/O but does not support VM live migration.

Consider the following when choosing a storage solution:

- ODF replicates data across multiple storage nodes for durability. This replication can cause unpredictable latency spikes during synchronization, which might affect **etcd** performance. Monitor **etcd** disk latency in production and ensure the underlying storage nodes use NVMe disks.

- LVM Storage writes directly to local disks without replication, providing consistent low latency. However, data is not replicated across nodes.

2.3. NETWORK REQUIREMENTS

Virtualized control plane deployments require specific network connectivity between VMs, worker nodes, and external services.

Configure networking to meet the following requirements:

- L2 network connectivity between control plane VMs. The OpenShift Container Platform installation program requires all control plane nodes to share the same L2 network segment.
- L3 connectivity without network address translation (NAT) between the control plane VMs and worker nodes. Worker nodes must be able to route traffic directly to the control plane VMs. During installation, the bootstrap VM also requires L3 connectivity to the control plane.
- External access to the cluster API.
- Access to container image registries.
- DNS resolution for cluster components.

You can use either dynamic or static IP addressing for the control plane VMs:

- For deployments that use DHCP, configure static MAC addresses on the primary network interfaces of the VMs and create DHCP reservations for each MAC address. This ensures consistent IP assignment across VM restarts.
- For deployments that use static IP addressing, define the network configuration in your **install-config.yaml** or Agent-based installation manifests.

2.4. CONTROL PLANE VM REQUIREMENTS

Each control plane VM must meet minimum resource requirements to run the OpenShift Container Platform control plane components reliably.

Each control plane VM requires the following minimum resources:

- 8 vCPUs
- 16 GiB RAM
- 120 GiB storage

2.5. KUBEVIRT REDFISH REQUIREMENTS

KubeVirt Redfish exposes VMs through the Redfish API.

You need the following to use KubeVirt Redfish:

- Network access from the installation workstation to the KubeVirt Redfish route.
- Credentials configured in KubeVirt Redfish for API authentication.

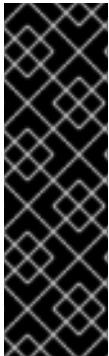
- VMs labeled to be exposed through the Redfish API.

2.6. ADDITIONAL RESOURCES

- [Recommended resources for the agent-based installer](#)
- [Effects of disk latency on etcd](#)

CHAPTER 3. PREPARING THE ENVIRONMENT FOR VIRTUALIZED CONTROL PLANES

Prepare your hosting cluster environment before deploying a virtualized control plane cluster. This includes installing and configuring KubeVirt Redfish and creating the control plane VMs.



IMPORTANT

KubeVirt Redfish is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

3.1. INSTALL KUBEVIRT REDFISH

Install KubeVirt Redfish on your OpenShift Virtualization cluster by applying a series of custom resources (CRs). These CRs create the namespace, permissions, configuration, and deployment required to expose VMs through the Redfish API.

Prerequisites

- You have a OpenShift Container Platform cluster with OpenShift Virtualization installed.
- You installed the OpenShift CLI (**oc**).
- You logged in to OpenShift Container Platform as a user with **cluster-admin** privileges.

Procedure

1. Create the **Namespace** CR for KubeVirt Redfish by creating a YAML file with content such as the following example:

```
apiVersion: v1
kind: Namespace
metadata:
  name: kubevirt-redfish
  labels:
    app.kubernetes.io/name: kubevirt-redfish
```

2. Apply the resource by running the following command:

```
$ oc apply -f namespace.yaml
```

3. Create the **ServiceAccount** CR by creating a YAML file with content such as the following example:

```
apiVersion: v1
kind: ServiceAccount
metadata:
```

```

name: kubevirt-redfish
namespace: kubevirt-redfish
labels:
  app.kubernetes.io/name: kubevirt-redfish
  app.kubernetes.io/component: rbac

```

4. Apply the resource by running the following command:

```
$ oc apply -f serviceaccount.yaml
```

5. Create the **ClusterRole** CR with required permissions by creating a YAML file with content such as the following example:

```

apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: kubevirt-redfish-role
  labels:
    app.kubernetes.io/name: kubevirt-redfish
    app.kubernetes.io/component: rbac
rules:
  - apiGroups: ["kubevirt.io"]
    resources: ["virtualmachines", "virtualmachineinstances"]
    verbs: ["get", "list", "watch", "update", "patch"]
  - apiGroups: ["kubevirt.io"]
    resources: ["virtualmachines/status", "virtualmachineinstances/status"]
    verbs: ["get", "list", "watch", "patch"]
  - apiGroups: ["kubevirt.io"]
    resources: ["virtualmachines/restart", "virtualmachines/start", "virtualmachines/stop"]
    verbs: ["create"]
  - apiGroups: ["subresources.kubevirt.io"]
    resources: ["virtualmachineinstances/pause", "virtualmachineinstances/unpause"]
    verbs: ["create", "update"]
  - apiGroups: [""]
    resources: ["pods", "services", "configmaps", "secrets"]
    verbs: ["get", "list", "watch", "create", "update", "delete"]
  - apiGroups: [""]
    resources: ["namespaces"]
    verbs: ["get", "list"]
  - apiGroups: ["cdi.kubevirt.io"]
    resources: ["datavolumes", "volumeimportsources"]
    verbs: ["get", "list", "watch", "create", "update", "patch", "delete"]
  - apiGroups: [""]
    resources: ["persistentvolumeclaims"]
    verbs: ["get", "list", "create", "update", "patch", "delete", "watch"]
  - apiGroups: ["storage.k8s.io"]
    resources: ["storageclasses"]
    verbs: ["get", "list"]

```

6. Apply the resource by running the following command:

```
$ oc apply -f clusterrole.yaml
```

7. Create the **ClusterRoleBinding** CR by creating a YAML file with content such as the following example:

```

apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: kubevirt-redfish-binding
  labels:
    app.kubernetes.io/name: kubevirt-redfish
    app.kubernetes.io/component: rbac
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: kubevirt-redfish-role
subjects:
  - kind: ServiceAccount
    name: kubevirt-redfish
    namespace: kubevirt-redfish

```

8. Apply the resource by running the following command:

```
$ oc apply -f clusterrolebinding.yaml
```

9. Create a **Secret** CR to enable the Redfish API to manage specific VMs:

- a. Generate a **bcrypt** hashed password for the Redfish API by running the following command in a Linux terminal:

```
$ htpasswd -nbB "" '<password>' | cut -d: -f2
```

- **<password>** is the password you want to use for the Redfish API.
The output is a **bcrypt** hashed password beginning with **\$2a\$**, **\$2b\$**, or **\$2y\$**.



NOTE

In earlier Technology Preview releases, plain text passwords were used. Use **bcrypt** hashed passwords for improved security.

- b. Create the **Secret** CR containing the configuration with content such as the following example. Edit the **config.yaml** section to match your environment:

```

apiVersion: v1
kind: Secret
metadata:
  name: kubevirt-redfish-secret
  namespace: kubevirt-redfish
  labels:
    app.kubernetes.io/name: kubevirt-redfish
    app.kubernetes.io/component: config
type: Opaque
stringData:
  config.yaml: |
    server:
      host: "0.0.0.0"
      port: 8443
      tls:
        enabled: false

```

```

system_id_convention: "enhanced"
chassis:
  - name: "<chassis_name>"
    namespace: "<vm_namespace>"
    service_account: "kubevirt-redfish"
    vm_selector:
      labels:
        redfish-enabled: "true"
authentication:
  users:
    - username: "admin"
      password:
        hash: "<bcrypt_password>"
        chassis: ["<chassis_name>"]
datavolume:
  storage_class: "<storage_class>"
  storage_size: "3Gi"

```

where:

system_id_convention

Specifies the format for Redfish system IDs. The recommended setting is **enhanced** to use **<namespace>.<vm-name>** format. The **legacy** setting uses **<vm-name>** only.

chassis

Specifies the namespaces where VMs are deployed. Replace **<chassis_name>** with a name for this chassis configuration and **<vm_namespace>** with the namespace containing your VMs. The **vm_selector** labels identify which VMs in the namespace are exposed through Redfish. Only VMs with matching labels are visible. You can configure multiple chassis entries to expose different subsets of VMs in the same namespace, each with different authentication users.

authentication

Specifies the username and password required to validate requests to the Redfish API. Replace **<bcrypt_password>** with the **bcrypt** hashed password that you generated.

datavolume

Specifies storage for VirtualMedia operations. Replace **<storage_class>** with a storage class available on your cluster, such as **lvms-vg1** or **ocs-storagecluster-ceph-rbd-virtualization**. For more information about storage options, see *Storage requirements* in "Prerequisites for virtualized control planes".

10. Apply the resource by running the following command:

```
$ oc apply -f secret.yaml
```

**WARNING**

The credentials defined in this **Secret** CR enable full management control over the VMs exposed through KubeVirt Redfish, independently of any OpenShift Container Platform privileges.

11. Create the **Deployment** CR by creating a YAML file with content such as the following example:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: kubevirt-redfish
  namespace: kubevirt-redfish
  labels:
    app.kubernetes.io/name: kubevirt-redfish
    app.kubernetes.io/component: server
spec:
  replicas: 1
  selector:
    matchLabels:
      app.kubernetes.io/name: kubevirt-redfish
      app.kubernetes.io/component: server
  template:
    metadata:
      labels:
        app.kubernetes.io/name: kubevirt-redfish
        app.kubernetes.io/component: server
    spec:
      serviceAccountName: kubevirt-redfish
      securityContext:
        runAsNonRoot: true
      containers:
        - name: kubevirt-redfish
          image: registry.redhat.io/container-native-virtualization/kubevirt-redfish-rhel9:v4.22
          imagePullPolicy: Always
          ports:
            - name: http
              containerPort: 8443
              protocol: TCP
          env:
            - name: CONFIG_PATH
              value: "/app/config/config.yaml"
            - name: LOG_LEVEL
              value: "info"
      resources:
        requests:
          memory: "512Mi"
          cpu: "100m"
        limits:
          memory: "2Gi"
          cpu: "500m"
```



```

livenessProbe:
  httpGet:
    path: /redfish/v1/
    port: 8443
    scheme: HTTP
  initialDelaySeconds: 30
  periodSeconds: 10
readinessProbe:
  httpGet:
    path: /redfish/v1/
    port: 8443
    scheme: HTTP
  initialDelaySeconds: 5
  periodSeconds: 5
securityContext:
  runAsNonRoot: true
  allowPrivilegeEscalation: false
  capabilities:
    drop:
      - ALL
volumeMounts:
  - name: config-volume
    mountPath: /app/config
    readOnly: true
volumes:
  - name: config-volume
    secret:
      secretName: kubevirt-redfish-secret

```

where:

- The **image** field specifies the KubeVirt Redfish container image.

12. Apply the resource by running the following command:

```
$ oc apply -f deployment.yaml
```

13. Create the **Service** CR by creating a YAML file with content such as the following example:

```

apiVersion: v1
kind: Service
metadata:
  name: kubevirt-redfish
  namespace: kubevirt-redfish
labels:
  app.kubernetes.io/name: kubevirt-redfish
  app.kubernetes.io/component: server
spec:
  type: ClusterIP
  ports:
    - name: http
      port: 8443
      targetPort: 8443
      protocol: TCP

```

```
selector:
  app.kubernetes.io/name: kubevirt-redfish
  app.kubernetes.io/component: server
```

14. Apply the resource by running the following command:

```
$ oc apply -f service.yaml
```

15. Create the **Route** CR to expose the Redfish API externally by creating a YAML file with content such as the following example:

```
apiVersion: route.openshift.io/v1
kind: Route
metadata:
  name: kubevirt-redfish
  namespace: kubevirt-redfish
labels:
  app.kubernetes.io/name: kubevirt-redfish
  app.kubernetes.io/component: server
spec:
  port:
    targetPort: http
  to:
    kind: Service
    name: kubevirt-redfish
    weight: 100
  tls:
    termination: edge
    insecureEdgeTerminationPolicy: Redirect
```

16. Apply the resource by running the following command:

```
$ oc apply -f route.yaml
```

Verification

1. Verify that the pods are running by running the following command:

```
$ oc get pods -n kubevirt-redfish
```

Example output

NAME	READY	STATUS	RESTARTS	AGE
kubevirt-redfish-587cd94988-xhtml	1/1	Running	0	2m

2. Get the route hostname by running the following command:

```
$ oc get route kubevirt-redfish -n kubevirt-redfish -o jsonpath='{.spec.host}'
```

3. Test the Redfish endpoint by running the following command:

```
$ curl -sk -u "admin:<password>" https://<route_hostname>/redfish/v1/
```

A successful response returns JSON with the Redfish service root:

```
{
  "@odata.id": "/redfish/v1",
  "@odata.type": "#ServiceRoot.v1_0_0.ServiceRoot",
  "Id": "RootService",
  "Name": "Root Service",
  "Systems": {
    "@odata.id": "/redfish/v1/Systems"
  }
}
```

3.2. CREATE CONTROL PLANE VMS

Create VMs on the hosting cluster that will become the control plane nodes for your virtualized control plane cluster.

Prerequisites

- KubeVirt Redfish is installed and configured on the hosting cluster.
- The hosting cluster has a network configured to provide Layer 2 connectivity between VMs.

Procedure

1. Enable the **RebootPolicy** feature gate on the hosting cluster by running the following command:

```
$ oc annotate --overwrite -n openshift-cnv hyperconverged kubevirt-hyperconverged \
kubevirt.kubevirt.io/jsonpatch='[{"op":"add","path":"/spec/configuration/developerConfiguration/f
eatureGates/-","value":"RebootPolicy"}]'
```



NOTE

The **RebootPolicy** feature gate enables the **rebootPolicy** field in **VirtualMachine** specifications. This configuration is required when using KubeVirt Redfish for cluster installation. The feature gate is enabled through an annotation on the **HyperConverged** resource, which propagates the configuration to the underlying **KubeVirt** CR.

2. Enable the **declarativeHotplugVolumes** feature gate on the hosting cluster by running the following command:

```
$ oc patch hyperconverged kubevirt-hyperconverged -n openshift-cnv \
--type merge \
-p '{"spec": {"featureGates": {"declarativeHotplugVolumes": true}}}'
```

**NOTE**

The **declarativeHotplugVolumes** feature gate enables KubeVirt Redfish to dynamically attach boot media to VMs through the Redfish API. This configuration is required when using KubeVirt Redfish for cluster installation.

3. Create a **VirtualMachine** CR for each control plane node by creating a YAML file with content such as the following example:

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: master-0
  namespace: <vm_namespace>
  labels:
    redfish-enabled: "true"
spec:
  runStrategy: Halted
  template:
    metadata:
      labels:
        redfish-enabled: "true"
    spec:
      domain:
        rebootPolicy: Terminate
        cpu:
          cores: 8
        memory:
          guest: 16Gi
        devices:
          disks:
            - name: rootdisk
              disk:
                bus: virtio
            - name: cloudinitdisk
              disk:
                bus: virtio
          interfaces:
            - name: default
              bridge: {}
        networks:
          - name: default
            multus:
              networkName: <network_attachment_definition>
      volumes:
        - name: rootdisk
          dataVolume:
            name: master-0-disk
        - name: cloudinitdisk
          cloudInitNoCloud:
            userData: |
              #cloud-config
              hostname: master-0
              user: core
```

where:

- **<vm_namespace>** specifies the namespace for the VMs. Must match the namespace specified in the KubeVirt Redfish chassis configuration.
- **redfish-enabled: "true"** specifies the label that must match the **vm_selector** labels in the KubeVirt Redfish configuration so the VM is exposed through the Redfish API.
- **runStrategy: Halted** specifies that VMs must be powered off initially. The installation powers them on by using the Redfish API.
- **rebootPolicy: Terminate** specifies the reboot behavior required for Redfish API boot override operations. Ensures the VM terminates cleanly when boot media changes.
- **cores: 8** and **guest: 16Gi** specify the minimum recommended resources for control plane nodes.
- **<network_attachment_definition>** specifies the name of a **NetworkAttachmentDefinition** configured on your hosting cluster. All control plane VMs must share the same L2 network segment. Common options include localnet, Linux bridge, or OVN Layer 2 networks.



IMPORTANT

For production deployments, configure anti-affinity rules to ensure control plane VMs are distributed across different physical nodes. This prevents a single node failure from affecting multiple control plane VMs simultaneously. Add pod anti-affinity rules or topology spread constraints to the VM specification based on your environment requirements.

4. Apply the resource by running the following command:

```
$ oc apply -f master-0.yaml
```

If required, create further VMs for **master-1** and **master-2**, for example.

Verification

- Verify that the VMs are created and powered off by running the following command:

```
$ oc get vm -n <vm_namespace>
```

- **vm_namespace** is the namespace of the VMs.

Example output

```
NAME    AGE  STATUS  READY
master-0 1m   Stopped False
master-1 1m   Stopped False
master-2 1m   Stopped False
```

- Verify that KubeVirt Redfish can discover the VMs by querying the Redfish API:

```
$ curl -sk -u "<username>:<password>" \
https://<kubevirt_redfish_route>/redfish/v1/Systems
```

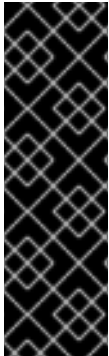


3.3. ADDITIONAL RESOURCES

- [Recommended resources for topologies](#)
- [Connecting a virtual machine to a secondary localnet user-defined network](#)

CHAPTER 4. DEPLOYING A VIRTUALIZED CONTROL PLANE

After preparing your environment, install the virtualized control plane cluster by using your preferred installation method. The agent-based installer and GitOps Zero Touch Provisioning (ZTP) are the recommended methods for virtualized control plane deployments.



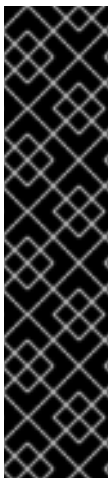
IMPORTANT

KubeVirt Redfish is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

4.1. DEPLOYING A VIRTUALIZED CONTROL PLANE WITH THE AGENT-BASED INSTALLER

Use the Agent-based Installer to deploy a virtualized control plane cluster. This method generates a bootable ISO containing all required configuration. You must mount the ISO to both the virtualized control plane nodes, by using KubeVirt Redfish, and the baremetal worker nodes, by using the Redfish API.



IMPORTANT

When configuring the Agent-based Installer, the **platform** parameter in **install-config.yaml** determines the level of hardware management:

platform: baremetal

Enables full hardware awareness. The cluster installs the Bare Metal Operator, and uses it to manage node lifecycle, power states, and automatic remediation through the Redfish API.

platform: none

Treats infrastructure as generic compute. The cluster cannot control power states or boot orders. Requires external DNS and load balancing.

Prerequisites

- KubeVirt Redfish is installed and configured on the hosting cluster.
- Control plane VMs are created on the hosting cluster and powered off. In this state, you can manage the VM power lifecycle and boot configuration through KubeVirt Redfish.
- You have network access to the KubeVirt Redfish route.
- An HTTP server is available to host the agent ISO.

Procedure

1. Create an **install-config.yaml** file with BMC addresses pointing to KubeVirt Redfish for the virtualized control plane nodes and real Redfish endpoints for the baremetal worker nodes:

```

apiVersion: v1
metadata:
  name: my-vcp-cluster
baseDomain: example.com
controlPlane:
  architecture: amd64
  hyperthreading: Enabled
  name: master
  replicas: 3
compute:
  - name: worker
    architecture: amd64
    hyperthreading: Enabled
    replicas: 2
networking:
  networkType: OVNKubernetes
  clusterNetwork:
    - cidr: 10.128.0.0/14
      hostPrefix: 23
  serviceNetwork:
    - 172.30.0.0/16
  machineNetwork:
    - cidr: 10.0.0.0/24
platform:
  baremetal:
    provisioningNetwork: Disabled
  apiVIPs:
    - 10.0.0.10
  ingressVIPs:
    - 10.0.0.11
  hosts:
    - name: master-0
      role: master
      bootMACAddress: 52:54:00:00:00:01
      bootMode: UEFI
      rootDeviceHints:
        deviceName: /dev/vda
      bmc:
        address: redfish-
virtualmedia+https://<kubevirt_redfish_route>/redfish/v1/Systems/<vm_namespace>.master-
0
      username: admin
      password: <password>
      disableCertificateVerification: false
  networkConfig:
    interfaces:
      - name: enp1s0
        mac-address: 52:54:00:00:00:01
        type: ethernet
        state: up
        ipv4:
          enabled: true
          dhcp: true
          auto-dns: true
        ipv6:
          enabled: true

```



```

- name: master-1
  role: master
  bootMACAddress: 52:54:00:00:00:02
  bootMode: UEFI
  rootDeviceHints:
    deviceName: /dev/vda
  bmc:
    address: redfish-
virtualmedia+https://<kubevirt_redfish_route>/redfish/v1/Systems/<vm_namespace>.master-
1
    username: admin
    password: <password>
    disableCertificateVerification: false
  networkConfig:
    interfaces:
      - name: enp1s0
        mac-address: 52:54:00:00:00:02
        type: ethernet
        state: up
        ipv4:
          enabled: true
          dhcp: true
          auto-dns: true
        ipv6:
          enabled: true
- name: master-2
  role: master
  bootMACAddress: 52:54:00:00:00:03
  bootMode: UEFI
  rootDeviceHints:
    deviceName: /dev/vda
  bmc:
    address: redfish-
virtualmedia+https://<kubevirt_redfish_route>/redfish/v1/Systems/<vm_namespace>.master-
2
    username: admin
    password: <password>
    disableCertificateVerification: false
  networkConfig:
    interfaces:
      - name: enp1s0
        mac-address: 52:54:00:00:00:03
        type: ethernet
        state: up
        ipv4:
          enabled: true
          dhcp: true
          auto-dns: true
        ipv6:
          enabled: true
- name: worker-0
  role: worker
  bootMACAddress: e4:43:4b:00:00:01
  bootMode: UEFI
  rootDeviceHints:
    deviceName: /dev/nvme0n1

```

```

bmc:
  address: redfish-
virtualmedia+https://<worker_bmc_ip>/redfish/v1/Systems/System.Embedded.1
  username: <bmc_username>
  password: <bmc_password>
  disableCertificateVerification: false
networkConfig:
  interfaces:
    - name: ens1f0
      mac-address: e4:43:4b:00:00:01
      type: ethernet
      state: up
      ipv4:
        enabled: true
        dhcp: true
        auto-dns: true
      ipv6:
        enabled: true
    - name: worker-1
      role: worker
      bootMACAddress: e4:43:4b:00:00:02
      bootMode: UEFI
      rootDeviceHints:
        deviceName: /dev/nvme0n1
      bmc:
        address: redfish-
virtualmedia+https://<worker_bmc_ip>/redfish/v1/Systems/System.Embedded.1
        username: <bmc_username>
        password: <bmc_password>
        disableCertificateVerification: false
      networkConfig:
        interfaces:
          - name: ens1f0
            mac-address: e4:43:4b:00:00:02
            type: ethernet
            state: up
            ipv4:
              enabled: true
              dhcp: true
              auto-dns: true
            ipv6:
              enabled: true
  pullSecret: '<pull_secret>'
  sshKey: '<ssh_public_key>'

```

where:

- **replicas** specifies the number of worker node replicas to match the number of baremetal worker hosts defined in the **hosts** section.
- **platform: baremetal** specifies full hardware lifecycle management. Set to **none** for infrastructure-agnostic installation.
- **provisioningNetwork: Disabled** specifies that the provisioning network is disabled for agent-based installations on virtual media. Nodes boot from the agent ISO, not PXE.

- **apiVIPs** and **ingressVIPs** specifies the virtual IP addresses that must be allocated from the machine network. The **apiVIPs** must reside on the same L2 network segment as the control plane VMs, and the **ingressVIPs** must reside on the same L2 network segment as the worker nodes.
- **bootMACAddress** specifies the MAC address used for network boot. When using DHCP, ensure this MAC has a reserved IP address configured in your DHCP server.
- **deviceName: /dev/vda** specifies the installation disk for virtualized control plane nodes.
- **bmc.address** for control plane nodes specifies the KubeVirt Redfish route. The `<vm_namespace>.<vm_name>` format corresponds to the enhanced system ID convention configured in KubeVirt Redfish. Replace `<kubevirt_redfish_route>` with your route hostname and `<vm_namespace>` with the namespace containing your VMs.
- **disableCertificateVerification** specifies whether to skip TLS certificate validation. For production deployments, configure properly signed TLS certificates and set to **false**. Set to **true** only for lab or development environments.
- **networkConfig** specifies the host network configuration for each node. This example uses DHCP.
- **deviceName: /dev/nvme0n1** specifies the installation disk for baremetal worker nodes.
- **bmc.address** for worker nodes specifies the real Redfish endpoint of the physical server. Replace `<worker_bmc_ip>` with the BMC IP address.

2. Create an **agent-config.yaml** YAML file similar to the following example:

```
apiVersion: v1alpha1
kind: AgentConfig
metadata:
  name: my-vcv-cluster
rendezvousIP: 10.0.0.20
```

where: * The **rendezvousIP** field specifies the IP address of the first control plane node. This node coordinates the installation.

3. Generate the agent ISO by running the following command:

```
$ openshift-install agent create image --dir <installation_directory>
```

4. Host the generated **agent.x86_64.iso** on an HTTP server accessible from your hosting cluster.

5. Boot each node from the agent ISO:

- For virtualized control plane nodes, use KubeVirt Redfish to mount the ISO and power on the VMs.
- For baremetal worker nodes, use the server's BMC interface to mount the ISO and boot.

6. Monitor the installation progress by running the following command:

```
$ openshift-install agent wait-for install-complete --dir <installation_directory>
```

- Replace **<installation_directory>** with the path to the directory where the agent ISO was generated.

Verification

- After installation completes, verify the cluster is operational:

```
$ export KUBECONFIG=<installation_directory>/auth/kubeconfig
$ oc get nodes
```

4.2. DEPLOYING A VIRTUALIZED CONTROL PLANE WITH GITOPS ZTP

Use GitOps Zero Touch Provisioning (ZTP) to deploy virtualized control plane clusters at scale. GitOps Zero Touch Provisioning (ZTP) uses GitOps to manage cluster deployments declaratively through Red Hat Advanced Cluster Management (RHACM).

Prerequisites

- RHACM is installed on a hub cluster.
- KubeVirt Redfish is installed and configured on the hosting cluster.
- Control plane VMs are created and powered off.
- A Git repository is configured for GitOps ZTP manifests.
- A pull secret is available for the cluster.

Procedure

1. Create **Secret** custom resources (CRs) that contain the BMC credentials for each node. The following example shows a secret for a control plane node:

```
apiVersion: v1
kind: Secret
metadata:
  name: master-0-bmc-secret
  namespace: my-vcp-cluster
type: Opaque
data:
  username: <base64_encoded_username>
  password: <base64_encoded_password>
```

Create similar **Secret** CRs for each node, for example **master-1-bmc-secret**, **worker-0-bmc-secret** and so on. For virtualized control plane nodes, use the KubeVirt Redfish credentials. For baremetal worker nodes, use the physical server's BMC credentials.

2. Create a **ClusterInstance** custom resource (CR) that defines the cluster. Specify BMC addresses pointing to KubeVirt Redfish for the virtualized control plane nodes and real Redfish endpoints for the baremetal worker nodes:

```
apiVersion: siteconfig.open-cluster-management.io/v1alpha1
kind: ClusterInstance
metadata:
```

```

name: my-vcp-cluster
namespace: my-vcp-cluster
spec:
  baseDomain: example.com
  clusterImageSetNameRef: "openshift-4.22"
  clusterName: my-vcp-cluster
  clusterType: HighlyAvailable
  platformType: BareMetal
  networkType: OVNKubernetes
  clusterNetwork:
    - cidr: 10.128.0.0/14
      hostPrefix: 23
  serviceNetwork:
    - cidr: 172.30.0.0/16
  machineNetwork:
    - cidr: 10.0.0.0/24
  apiVIPs:
    - 10.0.0.10
  ingressVIPs:
    - 10.0.0.11
  pullSecretRef:
    name: assisted-deployment-pull-secret
  sshPublicKey: "<ssh_public_key>"
  extraLabels:
    ManagedCluster:
      common: "true"
      sites: "my-vcp-cluster"
  templateRefs:
    - name: ai-cluster-templates-v1
      namespace: open-cluster-management
  nodes:
    - hostName: master-0.my-vcp-cluster.example.com
      role: master
      bmcAddress: redfish-
virtualmedia+https://<kubevirt_redfish_route>/redfish/v1/Systems/<vm_namespace>.master-
0
      bmcCredentialsName:
        name: master-0-bmc-secret
      bootMACAddress: 52:54:00:00:00:01
      bootMode: UEFI
      automatedCleaningMode: disabled
      rootDeviceHints:
        deviceName: /dev/vda
      templateRefs:
        - name: ai-node-templates-v1
          namespace: open-cluster-management
    - hostName: master-1.my-vcp-cluster.example.com
      role: master
      bmcAddress: redfish-
virtualmedia+https://<kubevirt_redfish_route>/redfish/v1/Systems/<vm_namespace>.master-
1
      bmcCredentialsName:
        name: master-1-bmc-secret
      bootMACAddress: 52:54:00:00:00:02
      bootMode: UEFI
      automatedCleaningMode: disabled

```

```

    rootDeviceHints:
      deviceName: /dev/vda
    templateRefs:
      - name: ai-node-templates-v1
        namespace: open-cluster-management
  - hostName: master-2.my-vcp-cluster.example.com
    role: master
    bmcAddress: redfish-
virtualmedia+https://<kubevirt_redfish_route>/redfish/v1/Systems/<vm_namespace>.master-
2
    bmcCredentialsName:
      name: master-2-bmc-secret
    bootMACAddress: 52:54:00:00:00:03
    bootMode: UEFI
    automatedCleaningMode: disabled
    rootDeviceHints:
      deviceName: /dev/vda
    templateRefs:
      - name: ai-node-templates-v1
        namespace: open-cluster-management
  - hostName: worker-0.my-vcp-cluster.example.com
    role: worker
    bmcAddress: redfish-
virtualmedia+https://<worker_bmc_ip>/redfish/v1/Systems/System.Embedded.1
    bmcCredentialsName:
      name: worker-0-bmc-secret
    bootMACAddress: e4:43:4b:00:00:01
    bootMode: UEFI
    automatedCleaningMode: disabled
    rootDeviceHints:
      deviceName: /dev/nvme0n1
    templateRefs:
      - name: ai-node-templates-v1
        namespace: open-cluster-management
  - hostName: worker-1.my-vcp-cluster.example.com
    role: worker
    bmcAddress: redfish-
virtualmedia+https://<worker_bmc_ip>/redfish/v1/Systems/System.Embedded.1
    bmcCredentialsName:
      name: worker-1-bmc-secret
    bootMACAddress: e4:43:4b:00:00:02
    bootMode: UEFI
    automatedCleaningMode: disabled
    rootDeviceHints:
      deviceName: /dev/nvme0n1
    templateRefs:
      - name: ai-node-templates-v1
        namespace: open-cluster-management

```

where:

- **apiVIPs** and **ingressVIPs** specifies the virtual IP addresses that must be allocated from the machine network. The **apiVIPs** must reside on the same L2 network segment as the control plane VMs, and the **ingressVIPs** must reside on the same L2 network segment as the worker nodes.

- **bmcAddress** for control plane nodes specifies the KubeVirt Redfish route. The `<vm_namespace>.<vm_name>` format corresponds to the enhanced system ID convention configured in KubeVirt Redfish. Replace `<kubevirt_redfish_route>` with your route hostname and `<vm_namespace>` with the namespace containing your VMs.
 - **bmcCredentialsName** specifies a reference to a **Secret** containing BMC credentials.
 - **bootMACAddress** specifies the MAC address used for network boot. When using DHCP, ensure this MAC has a reserved IP address configured in your DHCP server.
 - **deviceName: /dev/vda** specifies the installation disk for virtualized control plane nodes.
 - **bmcAddress** for worker nodes specifies the real Redfish endpoint of the physical server. Replace `<worker_bmc_ip>` with the BMC IP address.
 - **deviceName: /dev/nvme0n1** specifies the installation disk for baremetal worker nodes.
3. Commit the manifests to your Git repository.
 4. Apply the configuration through ArgoCD or your GitOps tooling.
ZTP generates the required resources and uses KubeVirt Redfish to provision the VMs automatically.

Verification

- Monitor the cluster deployment from the hub cluster:

```
$ oc get managedcluster my-vcp-cluster
$ oc get agentclusterinstall my-vcp-cluster -n my-vcp-cluster -o
jsonpath='{.status.debugInfo.stateInfo}'
```

4.3. DEPLOYING A VIRTUALIZED CONTROL PLANE WITH INSTALLER-PROVISIONED INFRASTRUCTURE

Use installer-provisioned infrastructure to deploy a virtualized control plane cluster. Installer-provisioned infrastructure provides full lifecycle management where the installation program automates hardware provisioning, power states, and cluster initialization.



NOTE

Installer-provisioned infrastructure for virtualized control planes requires a provisioning network or a RHEL 9 provisioner node. Verify Installer-provisioned infrastructure support for virtualized control plane deployments with your Red Hat representative, as this combination may have additional requirements.

Prerequisites

- KubeVirt Redfish is installed and configured on the hosting cluster.
- Control plane VMs are created on the hosting cluster and powered off. In this state, the installation manages the VM power lifecycle and boot configuration through KubeVirt Redfish.
- A RHEL 9 provisioner node is available, or a provisioning network is configured.

- You have network access to the KubeVirt Redfish route.

Procedure

1. Create the **install-config.yaml** file with BMC addresses pointing to KubeVirt Redfish for the virtualized control plane nodes and real Redfish endpoints for the baremetal worker nodes:

```

apiVersion: v1
metadata:
  name: my-vcv-cluster
baseDomain: example.com
controlPlane:
  architecture: amd64
  hyperthreading: Enabled
  name: master
  replicas: 3
  platform:
    baremetal: {}
compute:
  - name: worker
    architecture: amd64
    hyperthreading: Enabled
    replicas: 2
    platform:
      baremetal: {}
networking:
  networkType: OVNKubernetes
  clusterNetwork:
    - cidr: 10.128.0.0/14
      hostPrefix: 23
  serviceNetwork:
    - 172.30.0.0/16
  machineNetwork:
    - cidr: 10.0.0.0/24
platform:
  baremetal:
    apiVIPs:
      - 10.0.0.10
    ingressVIPs:
      - 10.0.0.11
    provisioningNetwork: Disabled
  hosts:
    - name: master-0
      role: master
      bootMACAddress: 52:54:00:00:00:01
      bootMode: UEFI
      rootDeviceHints:
        deviceName: /dev/vda
      bmc:
        address: redfish-
virtualmedia+https://<kubevirt_redfish_route>/redfish/v1/Systems/<vm_namespace>.master-
0
        username: admin
        password: <password>
        disableCertificateVerification: false
      networkConfig:

```



```

    interfaces:
      - name: enp1s0
        mac-address: 52:54:00:00:00:01
        type: ethernet
        state: up
        ipv4:
          enabled: true
          dhcp: true
          auto-dns: true
        ipv6:
          enabled: true
    - name: master-1
      role: master
      bootMACAddress: 52:54:00:00:00:02
      bootMode: UEFI
      rootDeviceHints:
        deviceName: /dev/vda
      bmc:
        address: redfish-
virtualmedia+https://<kubevirt_redfish_route>/redfish/v1/Systems/<vm_namespace>.master-
1
        username: admin
        password: <password>
        disableCertificateVerification: false
      networkConfig:
        interfaces:
          - name: enp1s0
            mac-address: 52:54:00:00:00:02
            type: ethernet
            state: up
            ipv4:
              enabled: true
              dhcp: true
              auto-dns: true
            ipv6:
              enabled: true
    - name: master-2
      role: master
      bootMACAddress: 52:54:00:00:00:03
      bootMode: UEFI
      rootDeviceHints:
        deviceName: /dev/vda
      bmc:
        address: redfish-
virtualmedia+https://<kubevirt_redfish_route>/redfish/v1/Systems/<vm_namespace>.master-
2
        username: admin
        password: <password>
        disableCertificateVerification: false
      networkConfig:
        interfaces:
          - name: enp1s0
            mac-address: 52:54:00:00:00:03
            type: ethernet
            state: up
            ipv4:

```

```

        enabled: true
        dhcp: true
        auto-dns: true
    ipv6:
        enabled: true
- name: worker-0
  role: worker
  bootMACAddress: e4:43:4b:00:00:01
  bootMode: UEFI
  rootDeviceHints:
    deviceName: /dev/nvme0n1
  bmc:
    address: redfish-
virtualmedia+https://<worker_bmc_ip>/redfish/v1/Systems/System.Embedded.1
    username: <bmc_username>
    password: <bmc_password>
    disableCertificateVerification: false
  networkConfig:
    interfaces:
      - name: ens1f0
        mac-address: e4:43:4b:00:00:01
        type: ethernet
        state: up
        ipv4:
          enabled: true
          dhcp: true
          auto-dns: true
        ipv6:
          enabled: true
- name: worker-1
  role: worker
  bootMACAddress: e4:43:4b:00:00:02
  bootMode: UEFI
  rootDeviceHints:
    deviceName: /dev/nvme0n1
  bmc:
    address: redfish-
virtualmedia+https://<worker_bmc_ip>/redfish/v1/Systems/System.Embedded.1
    username: <bmc_username>
    password: <bmc_password>
    disableCertificateVerification: false
  networkConfig:
    interfaces:
      - name: ens1f0
        mac-address: e4:43:4b:00:00:02
        type: ethernet
        state: up
        ipv4:
          enabled: true
          dhcp: true
          auto-dns: true
        ipv6:
          enabled: true
pullSecret: '<pull_secret>'
sshKey: '<ssh_public_key>'

```

where:

- **replicas** specifies the number of worker node replicas to match the number of baremetal worker hosts defined in the **hosts** section.
- **apiVIPs** and **ingressVIPs** specifies the virtual IP addresses that must be allocated from the machine network. The **apiVIPs** must reside on the same L2 network segment as the control plane VMs, and the **ingressVIPs** must reside on the same L2 network segment as the worker nodes.
- **provisioningNetwork: Disabled** specifies that the provisioning network is disabled when using virtual media.
- **bootMACAddress** specifies the MAC address used for network boot. When using DHCP, ensure this MAC has a reserved IP address configured in your DHCP server.
- **deviceName: /dev/vda** specifies the installation disk for virtualized control plane nodes.
- **bmc.address** for control plane nodes specifies the KubeVirt Redfish route. The **<vm_namespace>.<vm_name>** format corresponds to the enhanced system ID convention configured in KubeVirt Redfish. Replace **<kubevirt_redfish_route>** with your route hostname and **<vm_namespace>** with the namespace containing your VMs.
- **disableCertificateVerification** specifies whether to skip TLS certificate validation. For production deployments, configure properly signed TLS certificates and set to **false**. Set to **true** only for lab or development environments.
- **networkConfig** specifies the host network configuration for each node. This example uses DHCP.
- **deviceName: /dev/nvme0n1** specifies the installation disk for baremetal worker nodes.
- **bmc.address** for worker nodes specifies the real Redfish endpoint of the physical server. Replace **<worker_bmc_ip>** with the BMC IP address.

2. Run the installation by running the following command:

```
$ openshift-install create cluster --dir <installation_directory>
```

The installation process uses KubeVirt Redfish to manage VM power states and boot configuration automatically.

Verification

- After installation completes, verify the cluster is operational by running the following commands:

```
$ export KUBECONFIG=<installation_directory>/auth/kubeconfig
$ oc get nodes
```

4.4. ADDITIONAL RESOURCES

- [Preparing to install with the agent-based installer](#)
- [Installation configuration parameters for the agent-based installer](#)

- [Installation configuration parameters for bare metal](#)
- [Deploying far edge sites with ZTP](#)
- [BMC addressing for installer-provisioned infrastructure](#)
- [Configuring KubeVirt Redfish for VM management](#)
- [Preparing the provisioner node for OpenShift Container Platform installation](#)